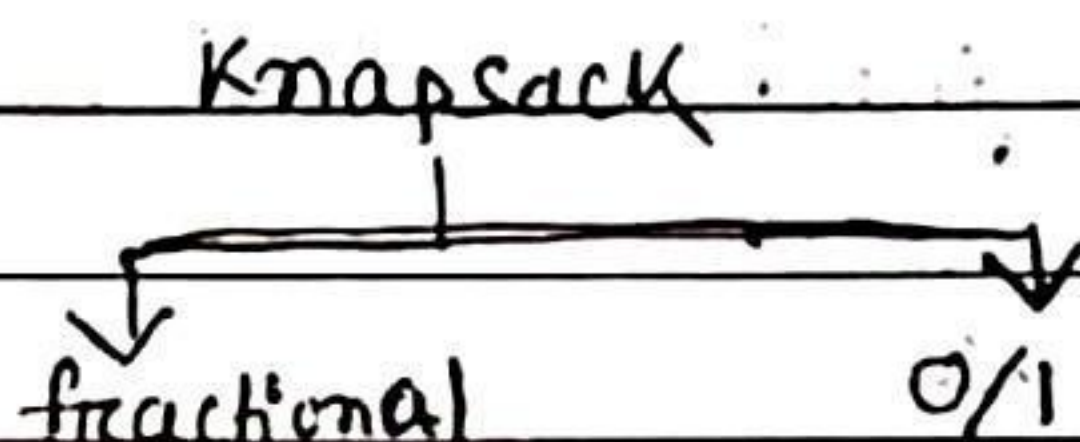


1/0 - Knapsack

• Difference between fractional and 0/1 knapsack —

• 0/1 Knapsack —

② ~~copy~~

Ex^{o1} Capacity = 6 units.

Objects —	1	2	3	find max profit.
Weight —	1	2	4	
profit —	10	12	28	

① Using Greedy method —

∅ p/w — 10 6 7

put the obj according to highest p/w ratio —

	1	→ 1 unit remaining, but we can't put 1 unit of obj object-2, because it is 0/1 knapsack.
P-28	} 4	
P-10	} 1	P = 38

Knapsack

here greedy method fails — (0/1 Knapsack)

28	} 4
102	} 2

Knapsack full and total profit = 40

Ex 2 why greedy method fails in 0-1 knapsack -

$[W > 1]$

object -	01	02	
profit -	2	W	
weight -	1	W	

capacity of knapsack = W.

$P/W = 2 \quad 1$

$W \left\{ \begin{array}{l} \text{ } \\ \text{ } \\ \text{ } \end{array} \right\} (W-1)$
 $\left\{ \begin{array}{l} \text{ } \\ \text{ } \\ P-2 \end{array} \right\} 1$

then we can't put second object. by ~~using~~ Thus why greedy method fails.

② Using Dynamic programming to solve 0-1 knapsack problem -

Objects - 1 2 3 4 ... n
 2 X 2 X 2 X 2 X 2 X 2 X - choice (either put or left along)

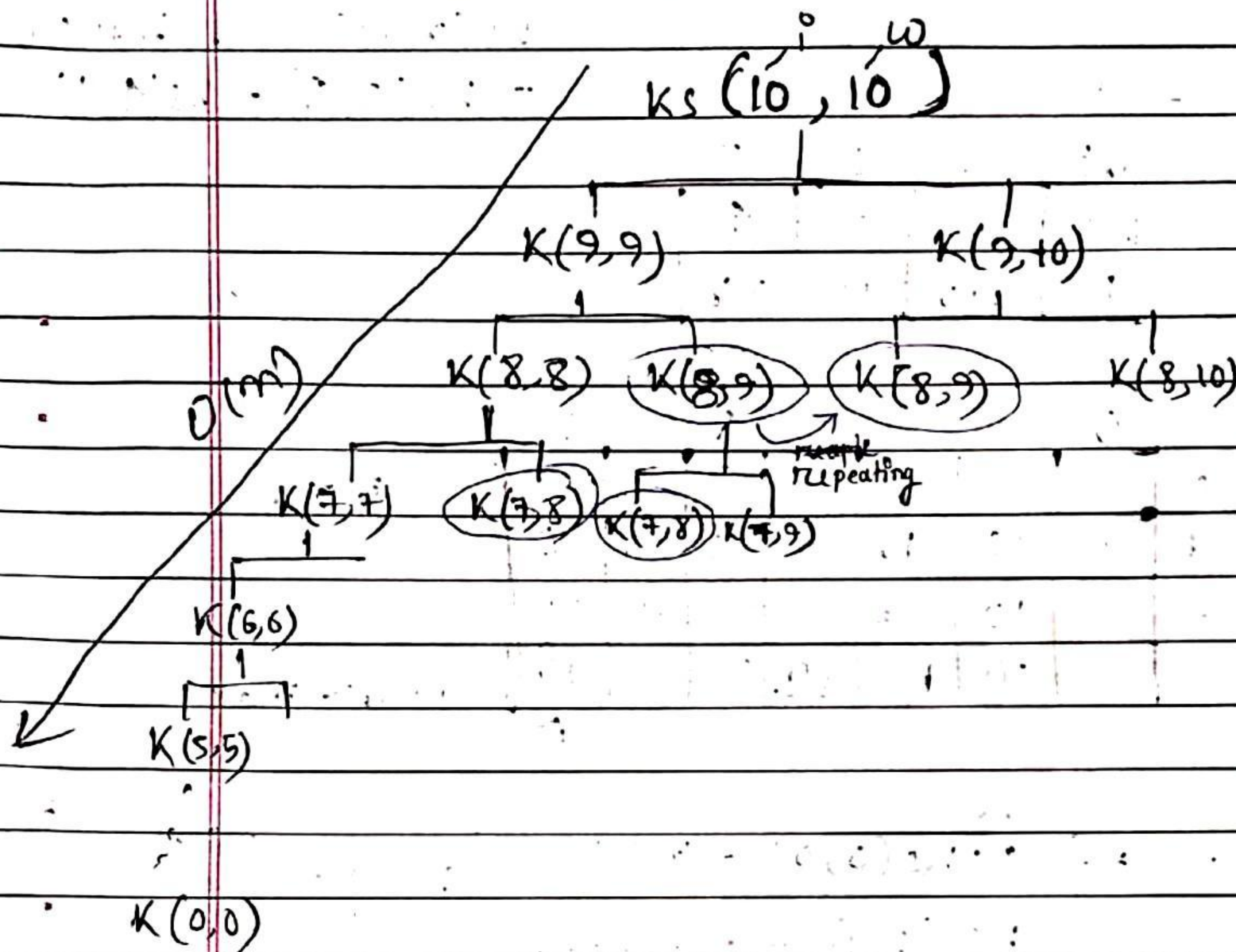
(2^n)

• Recursion equation -

$$KS(i, w) = \begin{cases} \max(P_i + KS(i-1, w - w_i), KS(i-1, w)) \\ 0 ; i=0 \text{ or } w=0 \\ KS(i-1, w) ; w_i > w \end{cases}$$

$i \rightarrow$ no. of obj
 $w \rightarrow$ capacity of knapsack

• Recursion tree -



total function call = $O(2^n)$ (if implement directly using recursive function)
 Space complexity = $O(m)$.

• Unique subproblems = $(m \times w)$
 $\Rightarrow 10 \times 10 = 100$

tabular method - (Dynamic Bottom up algo)
 ✓ Time can reduce from $O(2^n) \rightarrow O(mw)$

next page -

Example : 1 $C = 6$

(0, 0)

$$KS(n, c) = KS(n+1, c+1) \\ = KS(3, 6) = KS(4, 7)$$

Obj	1	2	3
W	1	2	4
P	10	12	28

	0	1	2	3	4	5	6
0	0	0	0	0	0	0	0
1	0	10	10	10	10	10	10
2	0	10	12	22	22	22	22
3	0	10	22	22	28	38	40

(RMO)

prefix max

$$KS(1,1) = \max \begin{cases} P_1 + KS(0,0) & \text{-- include} \\ KS(0,1) & \text{-- not include} \end{cases}$$

$$K(2,1) = \max \begin{cases} P_2 + K(1,0) \\ K(1,2) \end{cases}$$

$$KS(1,2) = \begin{cases} P_1 + K(0,1) = 10 \\ K(0,2) \end{cases}$$

$$K(2,2) = \begin{cases} P_2 + K(1,0) = 20 \\ K(1,2) = 10 \end{cases}$$

$$KS(1,3) = \begin{cases} P_1 + K(0,2) \\ K(0,3) \end{cases}$$

$$K(2,3) = \max \begin{cases} P_2 + K(1,1) \\ K(1,3) = 10 \end{cases}$$

$$K(2,4) = \begin{cases} P_2 + K(1,2) \\ K(1,4) = 10 \end{cases}$$

$$K(3,1) = \begin{cases} P_3 + K(2,0) \\ K(2,1) \end{cases}$$

$$K(3,4) = \begin{cases} P_3 + K(2,0) \\ K(2,4) = 23 \end{cases}$$

$$K(3,2) = \begin{cases} K(2,2) \end{cases}$$

$$K(3,3) = \begin{cases} K(2,3) \end{cases}$$

$$K(3,5) = \begin{cases} P_3 + K(2,1) \\ K(2,5) = 22 \end{cases}$$

$$K(3,6) = P_3 + K(2,2)$$

Time complexity = $O(nW)$

Space complexity = $O(nW)$

$$T(n) = \min \begin{pmatrix} O(2^n) \\ O(nW) \end{pmatrix}$$

• Algorithm for this -

Knapsack(i, W)

{

1. for $w = 0$ to W

2. if ($w_i > W$) $KS(0, W) = 0$

$p \rightarrow$ profit

Knapsack($i-1, W$)

3. for $i = 1$ to n

4. $KS(i, 0) = 0$

5. for $i = 1$ to n

6. for $w = 0$ to W

7. if $w_i \leq W$ // item i can be part of the solution.

8. if $p_i + KS[i-1, W-w_i] > KS[i-1, W]$

9. $KS(i, W) = p_i + KS[i-1, W-w_i]$

10. else

11. $KS[i, W] = KS[i-1, W]$

12. else

13. $KS[i, W] = KS[i-1, W]$ // $w_i > W$

}

[Signature]